



LeetCode 1456 — Maximum Number of Vowels in a Substring of Given Length

1. Problem Title & Link

Title: LeetCode 1456: Maximum Number of Vowels in a Substring of Given Length

Link: <https://leetcode.com/problems/maximum-number-of-vowels-in-a-substring-of-given-length/>

2. Problem Statement (Short Summary)

Given a string s and an integer k , find the maximum number of vowels present in any substring of length k .

A vowel is:

a, e, i, o, u

Return the maximum count of vowels found in any valid window of size k .

3. Examples (Input → Output)

Example 1

Input:

$s = \text{"abciidef"}$

$k = 3$

Output:

3

Explanation:

Substrings of length 3:

abc → 1 vowel

bci → 1 vowel

cii → 2 vowels

iii → 3 vowels

iid → 2 vowels

ide → 2 vowels

def → 1 vowel

Maximum = 3

Example 2

Input:

$s = \text{"aeiou"}$

$k = 2$

Output:



2

Example 3

Input:

s = "leetcode"

k = 3

Output:

2

4. Constraints

$1 \leq s.length \leq 100000$

s consists of lowercase English letters

$1 \leq k \leq s.length$

Important:

Length can be very large.

Need better than $O(n^2)$.

5. Core Concept (Pattern / Topic)

Fixed Sliding Window

This is one of the classic Sliding Window problems.

6. Thought Process (Step-by-Step Explanation)

Brute Force Approach

For every substring of size k:

1. Extract substring.
2. Count vowels.
3. Update maximum.

Example:

abciidef

abc

bci

cii

iii



...

For each window:

Count vowels again.

Time Complexity:

$O(n \times k)$

Too slow for large inputs.

Optimized Thinking

Notice:

Window size is fixed.

When moving from one window to the next:

Remove one character

Add one character

No need to recount the entire window.

Example

abc

Move right

bci

Only:

Remove 'a'

Add 'i'

Update count directly.

This is the Sliding Window idea.

7. Visual / Intuition Diagram (ASCII)

```
Input:
abciidef

k = 3
Window 1
[a b c]
Vowels = 1

Slide
a [b c i]
Remove:
a
```



```
Add:
i
New count remains:
1

Slide
a b [c i i]
Count becomes:
2
Continue similarly.
```

8. Pseudocode

```
vowels = {a,e,i,o,u}

count = vowels in first k characters

answer = count

for i from k to n-1

    remove left character

    add new character

    update count

    answer = max(answer, count)

return answer
```

9. Code Implementation

Python

```
class Solution:
    def maxVowels(self, s, k):

        vowels = {'a','e','i','o','u'}

        count = 0

        for i in range(k):
            if s[i] in vowels:
                count += 1

        ans = count

        for i in range(k, len(s)):
```



```
        if s[i-k] in vowels:
            count -= 1

        if s[i] in vowels:
            count += 1

        ans = max(ans, count)

    return ans
```

Java

```
import java.util.*;

class Solution {

    public int maxVowels(String s, int k) {

        HashSet<Character> vowels = new HashSet<> (
            Arrays.asList('a','e','i','o','u')
        );

        int count = 0;

        for(int i = 0; i < k; i++) {
            if(vowels.contains(s.charAt(i))) {
                count++;
            }
        }

        int ans = count;

        for(int i = k; i < s.length(); i++) {

            if(vowels.contains(s.charAt(i - k))) {
                count--;
            }

            if(vowels.contains(s.charAt(i))) {
                count++;
            }

            ans = Math.max(ans, count);
        }

        return ans;
    }
}
```



10. Time & Space Complexity

Time Complexity

$O(n)$

Single traversal.

Space Complexity

$O(1)$

Only five vowels stored.

11. Common Mistakes / Edge Cases

Mistake 1

Recounting vowels for every window.

$O(n \times k)$

Unnecessary.

Mistake 2

Forgetting to remove the outgoing character.

Mistake 3

Updating answer before adjusting the window.

Edge Case

$s = \text{"aeiou"}$

$k = 5$

Entire string is one window.

Answer:

5

12. Detailed Dry Run

Input:

$s = \text{"abciiidef"}$

$k = 3$

Initial Window:

abc

Vowels:

1

Answer:

1

Window	Count	Maximum
abc	1	1
bci	1	1
cii	2	2
iii	3	3
iid	2	3
ide	2	3
def	1	3

Final Answer:

3

13. Common Use Cases (Real-Life / Interview)

Fixed-size analytics windows:

Stock market trends

Website traffic analysis

Sensor readings

CPU monitoring

Streaming data

Example:

Maximum users in any 5-minute interval

Same pattern.

14. Common Traps (Important!)

Trap 1

Using substring extraction repeatedly.

Creates extra overhead.

Trap 2

Using nested loops.

Leads to:

$O(n \times k)$

Trap 3

Confusing Fixed Window and Variable Window.

This problem is:

Fixed Window

because k never changes.

15. Builds To (Related LeetCode Problems)

LC 643 Maximum Average Subarray I



LC 1456 Maximum Vowels



LC 567 Permutation in String



LC 438 Find All Anagrams



LC 76 Minimum Window Substring

Sliding Window Roadmap.

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n \times k)$	$O(1)$	Slow
Prefix Sum	$O(n)$	$O(n)$	Good
Sliding Window	$O(n)$	$O(1)$	Optimal

Approach 1: Brute Force

Count vowels for every substring.

Not efficient.

Approach 2: Prefix Sum

Store cumulative vowel counts.

Query each window quickly.

Works well.



Approach 3: Sliding Window

Maintain running count.

Best solution.

17. Why This Solution Works (Short Intuition)

When moving a fixed-size window:

One character leaves

One character enters

Instead of recalculating everything, we simply adjust the count.

Thus every window is processed in $O(1)$, leading to an overall $O(n)$ solution.

18. Variations / Follow-Up Questions

Follow-Up 1

Find minimum vowels in any window of size k .

Follow-Up 2

Find maximum consonants instead.

Follow-Up 3

Count distinct vowels in each window.

Follow-Up 4

Variable window size.

Leads to advanced Sliding Window problems.

Pattern Learned

Fixed Sliding Window

Build first window

↓

Slide one step

↓

Remove outgoing element

↓

Add incoming element

↓



Update answer